简易计算器项目学习指南

简易计算器项目学习指南 - 王力鑫

项目概述

本项目为使用 C 语言和 GTK 库开发的简易计算器应用程序，适合计算机专业大一新生作为 GUI 编程入门学习案例。该计算器具备基本的四则运算功能。

项目结构

Plain Text

```
simplecalculator/
├── .vscode/              # VSCode编辑器配置文件夹
│   └── c_cpp_properties.json  # C/C++语言配置文件
├── dlls/                 # 动态链接库文件
├── simplecalculator.c    # 主源代码文件
├── simplecalculator.exe # 编译后的可执行文件
├── run_calculator.bat    # 运行计算器的批处理文件
└── complete_gtk_dll_copy.sh # 复制GTK DLL的脚本
```

文件作用说明

1. **.vscode/c_cpp_properties.json**
   - VSCode 编辑器的 C/C++ 语言配置文件
   - 告诉 VSCode 编译器路径、包含文件路径等信息
   - 使 VSCode 能够正确识别和提示 GTK 库的函数和类型
   - 确保代码编辑时的语法高亮和智能提示功能正常工作
2. **dlls/** 目录
   - 包含项目依赖的动态链接库文件
   - DLL 文件是 Windows 系统中的动态链接库，类似于 Linux 中的 .so 文件
   - 这些文件包含了 GTK 库的实现代码，使我们的程序能够使用 GTK 的功能
   - 程序运行时会加载这些库文件
3. **simplecalculator.c**
   - 项目的主源代码文件

- 包含计算器的所有功能实现
- 定义了计算器的界面和逻辑

4. **simplecalculator.exe**
   - 编译后的可执行文件
   - 双击即可运行计算器程序
5. **run_calculator.bat**
   - Windows 批处理文件
   - 用于方便地运行计算器程序
   - 可以双击执行
6. **complete_gtk_dll_copy.sh**
   - Shell 脚本文件
   - 用于复制 GTK 库的 DLL 文件到指定位置
   - 在 MSYS2 终端中运行以执行该脚本：

```bash
bash complete_gtk_dll_copy.sh
```

- 注意，如果你只在本机进行软件的运行和调试，则不需要这个脚本，也不需要执行。这个脚本是为了解决在未安装 GTK 库的设备上运行本软件而编写的。它可以将软件依赖的 dll 文件复制到项目文件夹下，再搭配 **run_calculator.bat** 这个批处理文件，实现跨设备运行软件。

GTK 基础知识

什么是 GTK?

GTK（GIMP Toolkit）是一个跨平台的图形用户界面（GUI）工具包，最初为 GIMP 图像处理软件开发，现在被广泛用于各种应用程序的界面开发。

GTK 的基本概念

1. **窗口（Window）**：应用程序的主窗口
2. **控件（Widget）**：界面上的各种元素，如按钮、标签、输入框等
3. **布局容器（Container）**：用于组织和排列控件的容器，如 Box、Grid 等
4. **信号（Signal）**：事件通知机制，如按钮点击、窗口关闭等
5. **回调函数（Callback）**：响应信号的函数

GTK 的基本使用流程

1. 初始化 GTK 库
2. 创建主窗口
1. 创建各种控件和布局容器
2. 连接信号和回调函数
3. 显示所有控件
4. 进入主事件循环

GTK3 环境搭建（Windows + MinGW + MSYS2）

对于 Windows 系统，推荐使用 **MSYS2** 搭建 MinGW+GTK3 环境，步骤如下：

步骤 1：下载并安装 MSYS2

- 访问 MSYS2 官网：https://www.msys2.org/
- 下载安装包（如 `msys2-x86_64-xxx.exe`），双击安装
- 安装路径建议选择默认路径 `C:\msys64`（避免中文路径和空格）

步骤 2：更新 MSYS2 软件包

- 安装完成后，打开 **MSYS2 UCRT64** 终端
- 执行以下命令更新软件包数据库和系统：

```bash
pacman -Syu
```

- 若提示关闭终端，关闭后重新打开 **MSYS2 UCRT64** 终端再次执行：

```bash
pacman -Su
```

步骤 3：安装 MinGW-w64 工具链

- 在 **MSYS2 UCRT64** 终端中执行以下命令，安装 C/C++ 编译器和基础工具：

```bash
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-make mingw-w64-x86_64-pkg-config
```

步骤 4：安装 GTK3 库

- 在 **MSYS2 UCRT64** 终端中执行以下命令，安装 GTK3 及其开发文件：

```bash
pacman -S mingw-w64-ucrt-x86_64-gtk3
```

步骤 5：配置环境变量

- 右键点击“此电脑” → “属性” → “高级系统设置” → “环境变量”
- 在“系统变量”中找到 `Path`，点击“编辑”，添加以下路径（根据实际安装位置调整）：

```plain text
C:\msys64\ucrt64\bin
C:\msys64\usr\bin
C:\msys64\mingw64\bin
```

- 点击“确定”保存，关闭所有窗口

步骤 6：验证 GTK3 是否正确安装

- 在 **MSYS2 UCRT64** 终端中执行以下命令

```bash
pkg-config --modversion gtk+-3.0
```

- 若显示 gcc 版本和 GTK3 版本（如 `3.24.xx`），则环境搭建成功！

主程序代码结构

核心数据结构

Plain Text

```
// 计算器状态结构体
typedef struct {
    GtkWidget *display;     // 显示控件
    char current_number[256]; // 当前输入的数字
    double result;          // 计算结果
    char operation;         // 当前操作符
    int new_number;         // 是否开始输入新数字
} CalculatorState;
```

这个结构体用于存储计算器的状态信息，包括当前显示的内容、输入的数字、计算结果、选择的运算符等。

主要函数

1. **init_calculator_state**：初始化计算器状态
2. **update_display**：更新显示内容
3. **on_number_clicked**：处理数字按钮点击事件
4. **on_operator_clicked**：处理运算符按钮点击事件
5. **on_equals_clicked**：处理等号按钮点击事件
6. **on_clear_clicked**：处理清除按钮点击事件
7. **create_button**：创建按钮
8. **main**：主函数，程序入口

代码逻辑关系

1. **程序启动流程**：
    - 初始化 GTK 库
    - 创建主窗口
    - 初始化计算器状态
    - 创建布局容器和控件
    - 连接信号和回调函数
    - 显示所有控件
    - 进入主事件循环
2. **数字输入流程**：
    - 用户点击数字按钮
    - 触发 `on_number_clicked` 函数

- 更新 `current_number` 数组
- 调用 `update_display` 更新显示

3. **运算流程：**
   - 用户点击运算符按钮
   - 触发 `on_operator_clicked` 函数
   - 计算当前结果
   - 保存运算符
   - 准备输入新数字
   - 更新显示
4. **结果计算流程：**
   - 用户点击等号按钮
   - 触发 `on_equals_clicked` 函数
   - 计算最终结果
   - 更新显示
   - 重置运算符状态
5. **清除流程：**
   - 用户点击清除按钮
   - 触发 `on_clear_clicked` 函数
   - 重新初始化计算器状态
   - 更新显示

主程序函数模块分析

1. 核心数据结构

Plain Text

```
// 计算器状态结构体
typedef struct {
    GtkWidget *display;     // 显示控件
    char current_number[256]; // 当前输入的数字
    double result;          // 计算结果
    char operation;         // 当前操作符
    int new_number;         // 是否开始输入新数字
} CalculatorState;
```

分析：

- 这是一个结构体（struct），用于存储计算器的所有状态信息
- `display`：指向显示标签的指针，用于更新显示内容
- `current_number`：字符数组，存储用户当前输入的数字字符串
- `result`：双精度浮点数，存储计算结果
- `operation`：字符，存储当前选择的运算符
- `new_number`：整数，标记是否开始输入新数字（1 表示是，0 表示否）

2. 初始化函数

Plain Text

```
// 初始化计算器状态
void init_calculator_state(CalculatorState *state) {
    memset(state->current_number, 0, sizeof(state->current_number));
    state->result = 0;
    state->operation = '\0';
    state->new_number = 1;
}
```

分析：

- 函数功能：初始化计算器的状态
- 参数：`state` 是指向计算器状态结构体的指针
- `memset` 函数：将 `current_number` 数组的所有元素设置为 0（空字符）
- 将 `result` 设为 0，`operation` 设为空字符，`new_number` 设为 1（表示准备输入新数字）

3. 显示更新函数

Plain Text

```
// 更新显示
void update_display(CalculatorState *state) {
    gtk_label_set_text(GTK_LABEL(state->display), state->current_number);
}
```

分析：

- 函数功能：更新计算器的显示内容
- 参数：`state` 是指向计算器状态结构体的指针
- `gtk_label_set_text`：GTK 库函数，用于设置标签的文本内容
- `GTK_LABEL`：类型转换宏，将 `state->display` 转换为 `GtkLabel` 类型

4. 数字按钮点击事件处理函数

Plain Text

```
// 数字按钮点击事件
void on_number_clicked(GtkButton *button, gpointer data) {
    CalculatorState *state = (CalculatorState *)data;
    const char *text = gtk_button_get_label(button);

    if (state->new_number) {
        strcpy(state->current_number, text);
        state->new_number = 0;
    } else {
        strcat(state->current_number, text);
    }

    update_display(state);
}
```

分析：

- 函数功能：处理数字按钮的点击事件
- 参数：
  - `button`：指向被点击按钮的指针
  - `data`：传递给回调函数的数据（这里是计算器状态）
- 类型转换：将 `data` 转换为 `CalculatorState *` 类型
- 获取按钮标签：使用 `gtk_button_get_label` 获取按钮上的文本
- 逻辑处理：
  - 如果是新数字开始（`state->new_number` 为 1），则使用 `strcpy` 复制文本到 `current_number`
  - 否则，使用 `strcat` 将文本追加到 `current_number` 后面
  - 最后调用 `update_display` 更新显示

5. 运算符按钮点击事件处理函数

Plain Text

```
// 运算符按钮点击事件
void on_operator_clicked(GtkButton *button, gpointer data) {
    CalculatorState *state = (CalculatorState *)data;
    const char *text = gtk_button_get_label(button);

    if (strlen(state->current_number) > 0) {
        double current = atof(state->current_number);

        switch (state->operation) {
            case '+': state->result += current; break;
            case '-': state->result -= current; break;
            case '*': state->result *= current; break;
            case '/': state->result /= current; break;
            default: state->result = current; break;
        }

        state->operation = text[0];
        state->new_number = 1;

        // 显示当前结果
        snprintf(state->current_number, sizeof(state->current_number), "%.8g", state->result);
        update_display(state);
    }
}
```

分析：

- 函数功能：处理运算符按钮的点击事件
- 逻辑处理：
  - 首先检查 `current_number` 是否有内容
  - 使用 `atof` 函数将字符串转换为双精度浮点数
  - 根据当前的运算符执行相应的计算
  - 保存新的运算符
  - 设置 `new_number` 为 1，准备输入新数字
  - 使用 `snprintf` 将结果格式化为字符串
  - 调用 `update_display` 更新显示

6. 等号按钮点击事件处理函数

Plain Text

```
// 等号按钮点击事件
void on_equals_clicked(GtkButton *button, gpointer data) {
    (void)button;
    CalculatorState *state = (CalculatorState *)data;

    if (strlen(state->current_number) > 0) {
        double current = atof(state->current_number);

        switch (state->operation) {
            case '+': state->result += current; break;
            case '-': state->result -= current; break;
            case '*': state->result *= current; break;
            case '/': state->result /= current; break;
            default: state->result = current; break;
        }

        snprintf(state->current_number, sizeof(state->current_number), "%.8g", state->result);
        update_display(state);

        state->operation = '\0';
        state->new_number = 1;
    }
}
```

分析：

- 函数功能：处理等号按钮的点击事件，计算最终结果
- 逻辑处理：
  - `(void)button;`：表示不使用 `button` 参数，避免编译器警告
  - 检查 `current_number` 是否有内容
  - 将字符串转换为浮点数
  - 根据当前的运算符执行计算
  - 格式化结果并更新显示
  - 重置运算符为空字符，设置 `new_number` 为 1

7. 清除按钮点击事件处理函数

Plain Text

```
// 清除按钮点击事件
void on_clear_clicked(GtkButton *button, gpointer data) {
    (void)button;
    CalculatorState *state = (CalculatorState *)data;
    init_calculator_state(state);
    update_display(state);
}
```

分析：

- 函数功能：处理清除按钮的点击事件
- 逻辑处理：
  - 调用 `init_calculator_state` 重新初始化计算器状态
  - 调用 `update_display` 更新显示

8. 按钮创建函数

Plain Text

```
// 创建按钮
GtkWidget* create_button(const char *label, GCallback callback, gpointer data) {
    GtkWidget *button = gtk_button_new_with_label(label);
    g_signal_connect(button, "clicked", callback, data);
    return button;
}
```

分析：

- 函数功能：创建一个按钮并连接点击事件
- 参数：
  - `label`：按钮上的文本
  - `callback`：点击事件的回调函数
  - `data`：传递给回调函数的数据
- 返回值：创建的按钮控件
- 逻辑处理：
  - 使用 `gtk_button_new_with_label` 创建带有标签的按钮

- 使用 `g_signal_connect` 连接"clicked"信号和回调函数
- 返回创建的按钮

9. 主函数

Plain Text

```
int main(int argc, char *argv[]) {
    gtk_init(&argc, &argv);

    // 创建主窗口
    GtkWidget *window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_title(GTK_WINDOW(window), "简易计算器");
    gtk_window_set_default_size(GTK_WINDOW(window), 300, 400);
    gtk_container_set_border_width(GTK_CONTAINER(window), 10);

    // 初始化计算器状态
    CalculatorState state;
    init_calculator_state(&state);

    // 创建垂直布局容器
    GtkWidget *vbox = gtk_box_new(GTK_ORIENTATION_VERTICAL, 5);
    gtk_container_add(GTK_CONTAINER(window), vbox);

    // 创建显示标签
    state.display = gtk_label_new("0");
    gtk_widget_set_halign(state.display, GTK_ALIGN_END);
    gtk_box_pack_start(GTK_BOX(vbox), state.display, FALSE, FALSE, 5);

    // 创建按钮网格
    GtkWidget *grid = gtk_grid_new();
    gtk_grid_set_row_spacing(GTK_GRID(grid), 5);
    gtk_grid_set_column_spacing(GTK_GRID(grid), 5);
    gtk_box_pack_start(GTK_BOX(vbox), grid, TRUE, TRUE, 5);

    // 添加数字按钮
    for (int i = 0; i <= 9; i++) {
        char label[2];
        sprintf(label, "%d", i);
        GtkWidget *button = create_button(label, G_CALLBACK(on_number_clicked), &state);

        // 0按钮特殊处理（占两列）
        if (i == 0) {
            gtk_grid_attach(GTK_GRID(grid), button, 0, 4, 2, 1);
        } else {
            int row = 3 - (i - 1) / 3;
            int col = (i - 1) % 3;
            gtk_grid_attach(GTK_GRID(grid), button, col, row, 1, 1);
```

```
        }
    }

    // 添加运算符按钮
    GtkWidget *add_button = create_button("+", G_CALLBACK(on_operator_clicked), &state);
    gtk_grid_attach(GTK_GRID(grid), add_button, 3, 1, 1, 1);

    GtkWidget *sub_button = create_button("-", G_CALLBACK(on_operator_clicked), &state);
    gtk_grid_attach(GTK_GRID(grid), sub_button, 3, 2, 1, 1);

    GtkWidget *mul_button = create_button("*", G_CALLBACK(on_operator_clicked), &state);
    gtk_grid_attach(GTK_GRID(grid), mul_button, 3, 3, 1, 1);

    GtkWidget *div_button = create_button("/", G_CALLBACK(on_operator_clicked), &state);
    gtk_grid_attach(GTK_GRID(grid), div_button, 3, 4, 1, 1);

    // 添加等号按钮
    GtkWidget *equals_button = create_button("=", G_CALLBACK(on_equals_clicked), &state);
    gtk_grid_attach(GTK_GRID(grid), equals_button, 2, 4, 1, 1);

    // 添加清除按钮
    GtkWidget *clear_button = create_button("C", G_CALLBACK(on_clear_clicked), &state);
    gtk_grid_attach(GTK_GRID(grid), clear_button, 0, 0, 4, 1);

    // 显示所有控件
    gtk_widget_show_all(window);

    // 连接关闭事件
    g_signal_connect(window, "destroy", G_CALLBACK(gtk_main_quit), NULL);

    // 进入主循环
    gtk_main();

    return 0;
}
```

分析：

- 函数功能：程序的入口点，负责创建界面和启动程序
- 逻辑处理：
  a. 初始化 GTK 库：`gtk_init(&argc, &argv);`
  b. 创建主窗口并设置属性
  c. 初始化计算器状态
  d. 创建垂直布局容器
  e. 创建显示标签并添加到布局中
  f. 创建按钮网格并设置间距
  g. 使用循环创建数字按钮并添加到网格中
  h. 创建运算符按钮、等号按钮和清除按钮并添加到网格中
  i. 显示所有控件：`gtk_widget_show_all(window);`
  j. 连接窗口关闭事件
  k. 进入 GTK 主循环：`gtk_main();`
  l. 返回 0 表示程序正常结束

程序执行流程详解

1. 程序启动

1. 执行 `main` 函数
2. 初始化 GTK 库
3. 创建主窗口和布局
4. 初始化计算器状态
5. 创建并布局所有按钮
6. 显示所有控件
7. 进入主事件循环，等待用户交互

2. 用户输入数字

1. 点击数字按钮
2. 触发 `on_number_clicked` 函数
3. 获取按钮上的数字
4. 根据 `new_number` 状态决定是替换还是追加数字
5. 更新显示

3. 用户选择运算符

1. 点击运算符按钮
2. 触发 `on_operator_clicked` 函数
1. 获取当前输入的数字
2. 根据之前的运算符执行计算
3. 保存新的运算符
4. 显示计算结果
5. 准备输入新数字

4. 用户计算结果

1. 点击等号按钮
2. 触发 `on_equals_clicked` 函数
3. 获取当前输入的数字
4. 根据运算符执行最终计算
5. 显示计算结果
6. 重置运算符状态

5. 用户清除输入

1. 点击清除按钮
2. 触发 `on_clear_clicked` 函数
3. 重新初始化计算器状态
4. 更新显示为初始状态

代码解析

基础知识点

1. **结构体的使用**：
   - 结构体是 C 语言中一种自定义数据类型，可以包含多个不同类型的成员
   - 在本项目中，`CalculatorState` 结构体用于组织计算器的所有状态信息
2. **函数的定义和调用**：
   - 函数是 C 语言中实现特定功能的代码块
   - 本项目通过多个函数实现不同的功能，如初始化、更新显示、处理按钮点击等
3. **数组的使用**：
   - 数组是存储相同类型数据的集合
   - 在本项目中，`current_number` 数组用于存储用户输入的数字字符串

4. **条件语句**：
   - 如 `if` 语句和 `switch` 语句，用于根据不同条件执行不同的代码
   - 在处理运算符和计算结果时使用了 `switch` 语句
5. **循环语句**：
   - 如 `for` 循环，用于重复执行一段代码
   - 在创建数字按钮时使用了 `for` 循环
6. **指针和内存管理**：
   - 指针是存储内存地址的变量
   - 在本项目中，`gpointer data` 参数用于传递计算器状态的指针
7. **字符串处理**：
   - 如 `strcpy`、`strcat`、`sprintf` 等函数，用于处理字符串
   - 在处理用户输入和显示内容时使用了这些函数
8. **GTK 界面编程**：
   - 创建窗口、控件和布局
   - 连接信号和回调函数
   - 显示控件和进入事件循环

代码亮点

1. **模块化设计**：
   - 将不同功能封装为独立函数，使代码结构清晰
   - 例如，`create_button` 函数专门用于创建按钮
2. **状态管理**：
   - 使用结构体统一管理计算器状态
   - 便于在不同函数间传递和修改状态
3. **事件驱动编程**：
   - 通过 GTK 的信号机制处理用户交互
   - 当用户点击按钮时，触发相应的回调函数
4. **网格布局**：
   - 使用 GTK Grid 组件实现按钮的网格排列
   - 使界面布局整齐美观

运行和调试

方法 1：直接运行（仅适合已完整部署 DLL 的情况）

双击 `simplecalculator.exe` 文件（**注意**：若未将 `dlls` 目录下的所有 GTK DLL 复制到与 `.exe` 同级，或未临时 / 永久配置环境变量，此操作会报错“找不到 xxx.dll”）。

方法 2：使用批处理文件（Windows 临时配置环境变量）

双击 `run_calculator.bat` 文件，脚本会自动临时配置环境变量并启动程序，无需手动复制所有 DLL（通过 `complete_gtk_dll_copy.sh` 提前生成）老师提供的项目中以完整包含了核心 GTK DLL，通过此方法可以直接运行。

方法 3：命令行编译（基础编译方法，较复杂）

如果需要修改代码并重新编译，先确保已完成 **“GTK3 环境搭建（Windows + MinGW + MSYS2）”** 部分，然后按以下步骤操作：

1. 打开 **MSYS2 UCRT64** 终端。
2. 进入项目所在目录（假设项目在桌面的 `simplecalculator` 文件夹）：

```bash
cd /c/Users/你的用户名/Desktop/simplecalculator  # 替换为你的实际路径
```

小提示：如果不知道路径怎么写，可以在文件夹地址栏输入 `cmd` 或 `powershell` 打开终端，输入 `pwd`（PowerShell）或 `cd`（CMD）查看路径，再把反斜杠 `\` 改成正斜杠 `/`，盘符前加 `/`（如 `C:\Users` 改成 `/c/Users`）。

3. 执行编译命令：

```bash
gcc -o simplecalculator.exe simplecalculator.c `pkg-config --cflags --libs gtk+-3.0`
```

小提示：

- `gcc`：调用 C 语言编译器。
- `-o simplecalculator.exe`：指定输出的可执行文件名为 `simplecalculator.exe`。
- `simplecalculator.c`：要编译的源代码文件。
- `` `pkg-config --cflags --libs gtk+-3.0` ``：**反引号（键盘左上角 Esc 键下方）包裹的命令**，作用是自动告诉编译器 GTK3 的头文件路径（`--cflags`）和库文件路径 / 名称（`--libs`），新手**千万不要改反引号或里面的内容**！

4. 编译成功后，终端不会有任何提示（或只有极少量警告，不影响运行）；如果报错，先检查代码语法、环境变量是否配置正确、终端是否是 **MSYS2 UCRT64**。

方法 4：VSCode 编译与调试（推荐使用）

VSCode 是微软官方免费的轻量级代码编辑器，配合插件可以实现“写代码→一键编译→一键调试→看结果”的全流程，非常适合新手。

VSCode 编译前的准备（按顺序完成）

准备 1：安装 VSCode

1. 访问 VSCode 官网：https://code.visualstudio.com/
2. 点击 **“Download for Windows”** 下载稳定版安装包。
3. 双击安装包，按提示安装：
   - 安装路径建议选择 `C:\Program Files\Microsoft VS Code`（默认即可，不要有中文路径和空格）。
   - 在“选择附加任务”页面，**务必勾选以下 4 项**：

✅ 将“通过 Code 打开”操作添加到 Windows 资源管理器文件上下文菜单

✅ 将“通过 Code 打开”操作添加到 Windows 资源管理器目录上下文菜单

✅ 将 Code 注册为受支持的文件类型的编辑器

✅ 添加到 PATH（重启后生效）

   - 其他选项默认即可，点击“安装”→“完成”。

准备 2：安装 VSCode 中文语言包

1. 打开 VSCode，点击左侧边栏的 **“扩展”图标**（或按快捷键 `Ctrl+Shift+X`）。
2. 在搜索框输入 `Chinese (Simplified) (简体中文) Language Pack for Visual Studio Code`。
3. 点击第一个搜索结果（微软官方的，图标是红色的“中”字）的 **“安装”** 按钮。
4. 安装完成后，VSCode 右下角会弹出提示“是否重启以应用语言包？”，点击 **“Restart”** 重启。

准备 3：安装 VSCode 必要的 C/C++ 插件

1. 重启后，再次点击左侧边栏的 **“扩展”图标**（或按 `Ctrl+Shift+X`）。
2. 在搜索框输入 `C/C++ Extension Pack`。
3. 点击第一个搜索结果的 **“安装”** 按钮。

小提示：这个扩展包包含了新手需要的所有 C/C++ 相关插件：

- C/C++：核心插件，提供语法高亮、智能提示、编译 / 调试配置功能。
- C/C++ Themes：C/C++ 代码主题（可选，新手可以不用管）。
- CMake Tools：CMake 构建工具（暂时不用，以后学大型项目再用）。
- Doxygen Documentation Generator：自动生成代码注释（暂时不用）。

准备 4：配置 VSCode 的默认终端为 MSYS2 MinGW 64-bit

这是**最关键的一步**！VSCode 默认的终端是 PowerShell/CMD，找不到我们之前安装的 `gcc` 和 `pkg-config`，必须改成 MSYS2 的 MinGW64 终端。

1. 打开 VSCode，按快捷键 `Ctrl+,`（逗号）打开 **“设置”** 页面。
2. 在设置页面顶部的搜索框输入 `terminal.integrated.profiles.windows`。
3. 点击搜索结果下方的 **“在 settings.json 中编辑”** 按钮（注意：不要直接在输入框里改，必须打开 `settings.json`）。
4. 在打开的 `settings.json` 文件中，**找到大括号 `{}` 包裹的内容**，在里面**添加以下代码**（如果大括号里已经有其他内容，记得在最后加一个逗号 `,` 再粘贴）：

JSON

```
// 配置 MSYS2 MinGW 64-bit 终端
"terminal.integrated.profiles.windows": {
    "MSYS2 MinGW 64-bit": {
        "path": "C:\\msys64\\usr\\bin\\bash.exe",  // MSYS2的bash路径，装在D盘就改成D:\\msys64\\...
        "args": [
            "--login",
            "-i",
            "-c",
            "export MSYSTEM=MINGW64; export PATH=/mingw64/bin:$PATH; exec bash"
        ],
        "icon": "terminal-bash"
    }
},
// 设置默认终端为 MSYS2 MinGW 64-bit
"terminal.integrated.defaultProfile.windows": "MSYS2 MinGW 64-bit"
```

小提示：

- 代码里的 `\\` 是转义的反斜杠，**必须写两个**，不能写一个！
- 如果你的 MSYS2 不是装在 `C:\msys64`，比如装在 `D:\Softwares\msys64`，就把 `path` 改成 `"D:\\Softwares\\msys64\\usr\\bin\\bash.exe"`。

5. 按快捷键 `Ctrl+S` 保存 `settings.json` 文件，然后关闭文件。
6. 验证终端是否配置成功：
   - 按快捷键 `Ctrl+~` （波浪号，键盘左上角 Esc 键下方）打开 VSCode 的终端。
   - 在终端里输入 `gcc --version` 和 `pkg-config --modversion gtk+-3.0`，如果分别显示 gcc 版本和 GTK3 版本（如 `3.24.xx`），说明配置成功！

VSCode 一键编译 / 调试方法

第一步：用 VSCode 打开项目文件夹

1. 打开 VSCode，点击左上角的 **“文件”** → **“打开文件夹”**（或按快捷键 `Ctrl+K` 松开后按 `Ctrl+O`）。
2. 选择你的 `simplecalculator` 项目文件夹，点击 **“选择文件夹”**。

小提示：打开后，左侧边栏的 **“资源管理器”**（或按 `Ctrl+Shift+E`）会显示项目的所有文件，点击 `simplecalculator.c` 就能打开代码编辑。

第二步：自动生成 / 手动配置编译和调试文件

VSCode 需要两个配置文件才能实现一键编译 / 调试：

- `.vscode/tasks.json`：告诉 VSCode 怎么编译代码（对应之前的命令行编译命令）。
- `.vscode/launch.json`：告诉 VSCode 怎么调试代码（设置断点、单步执行等）。

本项目提供的项目文件夹里已经有 `.vscode` 文件夹和 `c_cpp_properties.json`、`tasks.json` 和 `launch.json` 三个配置文件，请尝试阅读，并借助 AI 分析这几个配置文件都做了什么？

如果你是自己创建新项目，上述的配置文件都需要自己**手动添加**，否则是无法直接在 VScode 里进行编译的。

手动添加配置文件方法（本项目已经提供了配置文件，下面的操作仅供同学了解）

1. 在左侧边栏的 **“资源管理器”** 中，右键点击 `.vscode` 文件夹（如果没有，就右键点击项目文件夹 `simplecalculator` → **“新建文件夹”**，命名为 `.vscode`，注意前面有个点）。
2. 点击 **“新建文件”**，命名为 `tasks.json`，按回车。
3. 把以下代码**完整复制**到 `tasks.json` 中，按 `Ctrl+S` 保存：

JSON

```
{
  "version": "2.0.0",
  "tasks": [
    {
        "type": "cppbuild",
        "label": "MSYS2_64 C/C++: gcc.exe build active file",
        "command": "C:\\msys64\\mingw64\\bin\\gcc.exe",
        "args": [
            "-g",
            "-Wall",
            "-Wextra",
            "${file}",
            "-o",
            "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "-lm",
            "-fdiagnostics-color=always",
            "-IC:/msys64/mingw64/include/gtk-3.0",
            "-IC:/msys64/mingw64/include/pango-1.0",
            "-IC:/msys64/mingw64/include",
            "-IC:/msys64/mingw64/include/glib-2.0",
            "-IC:/msys64/mingw64/lib/glib-2.0/include",
            "-IC:/msys64/mingw64/include/harfbuzz",
            "-IC:/msys64/mingw64/include/freetype2",
            "-IC:/msys64/mingw64/include/libpng16",
            "-IC:/msys64/mingw64/include/fribidi",
            "-IC:/msys64/mingw64/include/cairo",
            "-IC:/msys64/mingw64/include/lzo",
            "-IC:/msys64/mingw64/include/pixman-1",
            "-mms-bitfields",
            "-IC:/msys64/mingw64/include/gdk-pixbuf-2.0",
            "-IC:/msys64/mingw64/include/atk-1.0",
            "-lgtk-3",
            "-lgdk-3",
            "-lz",
            "-lgdi32",
            "-limm32",
            "-lshell32",
            "-lole32",
            "-luuid",
            "-lwinmm",
            "-ldwmapi",
            "-lsetupapi",
```

```
            "-lcfgmgr32",
            "-lpangowin32-1.0",
            "-lpangocairo-1.0",
            "-lpango-1.0",
            "-lharfbuzz",
            "-latk-1.0",
            "-lcairo-gobject",
            "-lcairo",
            "-lgdk_pixbuf-2.0",
            "-lgio-2.0",
            "-lgobject-2.0",
            "-lglib-2.0",
            "-lintl"
        ],
        "options": {
            "cwd": "C:\\msys64\\mingw64\\bin"
        },
        "problemMatcher": [
            "$gcc"
        ],
        "detail": "compiler: C:\\msys64\\mingw64\\bin\\gcc.exe",
        "group": "build"
    },
    {
        "type": "cppbuild",
        "label": "MSYS2 GCC_10 C/C++: g++.exe build active file",
        "command": "C:\\msys64\\mingw64\\bin\\g++.exe",
        "args": [
            "-g",
            "-Wall",
            "-Wextra",
            "-mms-bitfields",
            "${file}",
            "-o",
            "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "-lm",
            "-lole32",
            "-lkernel32",
            "-lucrt",
            "-D_FILE_OFFSET_BITS=64",
            "-D_LARGEFILE64_SOURCE",
            "-fdiagnostics-color=always",
            "-IC:/msys64/mingw64/include/gtk-3.0",
            "-IC:/msys64/mingw64/include/pango-1.0",
```

```
            "-IC:/msys64/mingw64/include",
            "-IC:/msys64/mingw64/include/glib-2.0",
            "-IC:/msys64/mingw64/lib/glib-2.0/include",
            "-IC:/msys64/mingw64/include/harfbuzz",
            "-IC:/msys64/mingw64/include/freetype2",
            "-IC:/msys64/mingw64/include/libpng16",
            "-IC:/msys64/mingw64/include/fribidi",
            "-IC:/msys64/mingw64/include/cairo",
            "-IC:/msys64/mingw64/include/lzo",
            "-IC:/msys64/mingw64/include/pixman-1",
            "-mms-bitfields",
            "-IC:/msys64/mingw64/include/gdk-pixbuf-2.0",
            "-IC:/msys64/mingw64/include/atk-1.0",
            "-lgtk-3",
            "-lgdk-3",
            "-lz",
            "-lgdi32",
            "-limm32",
            "-lshell32",
            "-lole32",
            "-luuid",
            "-lwinmm",
            "-ldwmapi",
            "-lsetupapi",
            "-lcfgmgr32",
            "-lpangowin32-1.0",
            "-lpangocairo-1.0",
            "-lpango-1.0",
            "-lharfbuzz",
            "-latk-1.0",
            "-lcairo-gobject",
            "-lcairo",
            "-lgdk_pixbuf-2.0",
            "-lgio-2.0",
            "-lgobject-2.0",
            "-lglib-2.0",
            "-lintl"
        ],
        "options": {
            "cwd": "C:\\msys64\\mingw64\\bin"
        },
        "problemMatcher": [
            "$gcc"
        ],
```

```
        "detail": "compiler: C:\\msys64\\mingw64\\bin\\g++.exe",
        "group": {
            "kind": "build",
            "isDefault": true
        }
    },
    {
        "type": "cppbuild",
        "label": "Release Build (Static)",
        "command": "C:\\msys64\\mingw64\\bin\\gcc.exe",
        "args": [
            "-O2",                      // 优化级别2
            "-mwindows",                // 隐藏控制台窗口
            "-Wall",
            "-Wextra",
            "-mms-bitfields",
            "${file}",
            "-o",
            "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "-lm",
            "-lole32",
            "-lkernel32",
            "-lucrt",
            "-D_FILE_OFFSET_BITS=64",
            "-D_LARGEFILE64_SOURCE",
            "-fdiagnostics-color=always",
            "-IC:/msys64/mingw64/include/gtk-3.0",
            "-IC:/msys64/mingw64/include/pango-1.0",
            "-IC:/msys64/mingw64/include",
            "-IC:/msys64/mingw64/include/glib-2.0",
            "-IC:/msys64/mingw64/lib/glib-2.0/include",
            "-IC:/msys64/mingw64/include/harfbuzz",
            "-IC:/msys64/mingw64/include/freetype2",
            "-IC:/msys64/mingw64/include/libpng16",
            "-IC:/msys64/mingw64/include/fribidi",
            "-IC:/msys64/mingw64/include/cairo",
            "-IC:/msys64/mingw64/include/lzo",
            "-IC:/msys64/mingw64/include/pixman-1",
            "-IC:/msys64/mingw64/include/gdk-pixbuf-2.0",
            "-IC:/msys64/mingw64/include/atk-1.0",
            "-lgtk-3",
            "-lgdk-3",
            "-lz",
            "-lgdi32",
```

```
            "-limm32",
            "-lshell32",
            "-lole32",
            "-luuid",
            "-lwinmm",
            "-ldwmapi",
            "-lsetupapi",
            "-lcfgmgr32",
            "-lpangowin32-1.0",
            "-lpangocairo-1.0",
            "-lpango-1.0",
            "-lharfbuzz",
            "-latk-1.0",
            "-lcairo-gobject",
            "-lcairo",
            "-lgdk_pixbuf-2.0",
            "-lgio-2.0",
            "-lgobject-2.0",
            "-lglib-2.0",
            "-lintl"
        ],
        "options": {
            "cwd": "C:\\msys64\\mingw64\\bin"
        },
        "problemMatcher": [
            "$gcc"
        ],
        "detail": "Release build with optimization and no console window",
        "group": {
            "kind": "build",
            "isDefault": true
        }
    }
]
}
```

小提示：

- `-g`：**必须保留，**作用是生成调试信息，没有这个就不能用断点调试。
- `isDefault: true`：设置为默认编译任务，以后按 `Ctrl+Shift+B` 就能直接编译，不用再选任务。

4. 再次右键点击 `.vscode` 文件夹 → **“新建文件”**，命名为 `launch.json`，按回车。
5. 把以下代码**完整复制**到 `launch.json` 中，按 `Ctrl+S` 保存：

JSON

```json
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Debug GTK Program",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/simplecalculator.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [
                {
                    "name": "PATH",
                    "value": "C:/msys64/mingw64/bin;${env:PATH}"
                }
            ],
            "externalConsole": true,
            "MIMode": "gdb",
            "miDebuggerPath": "C:/msys64/mingw64/bin/gdb.exe",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

小提示：

- `externalConsole: true`：**必须保留，**GTK 程序需要弹出独立的窗口，不能在 VSCode 的内置终端里运行。
- `miDebuggerPath`：如果你的 MSYS2 不是装在 `C:\msys64`，记得改路径，这里用正斜杠 `/` 或转义的反斜杠 `\\` 都可以。

第三步：一键编译

按快捷键 `Ctrl+Shift+B`，VSCode 会自动调用 `tasks.json` 里的默认编译任务编译代码。

- 编译成功：终端会显示“终端将被任务重用，按任意键关闭”，没有其他错误提示。
- 编译失败：终端会显示具体的错误信息（比如哪一行代码语法错了），点击错误信息可以直接跳转到代码的错误位置，修改后再按 `Ctrl+Shift+B` 编译。

第四步：一键调试（新手可以先学断点调试）

1. **设置断点**：在代码编辑区，点击你想暂停的行号前面的空白处（比如 `main` 函数的第一行 `gtk_init(&argc, &argv);` 前面），会出现一个**红色的圆点**，这就是断点。

小提示：断点的作用是让程序运行到这一行时自动暂停，你可以查看此时的变量值、单步执行代码，方便找 bug。

2. **启动调试**：按快捷键 `F5`，或者点击左侧边栏的 **“运行和调试”图标**（或按 `Ctrl+Shift+D`），再点击顶部的绿色 **“播放按钮”**。
3. **调试操作**：程序启动后会自动暂停在第一个断点处，此时顶部会出现一个**调试工具栏**，常用的按钮有：
   - **继续（F5）**：继续运行程序，直到遇到下一个断点或程序结束。
   - **单步跳过（F10）**：执行当前行代码，然后暂停在下一行（如果当前行是函数调用，不会进入函数内部）。
   - **单步进入（F11）**：执行当前行代码，如果当前行是函数调用，会进入函数内部暂停。
   - **单步跳出（Shift+F11）**：从当前函数内部跳出，暂停在函数调用的下一行。
   - **停止（Shift+F5）**：停止调试，关闭程序。
4. **查看变量值**：程序暂停时，左侧边栏的 **“运行和调试”** 页面会显示当前作用域内的所有变量值，你也可以把鼠标悬停在代码中的变量名上查看值。

第五步：一键运行（不需要调试时）

如果不需要断点调试，只想快速运行程序，可以：

1. 先按 `Ctrl+Shift+B` 编译代码（如果已经编译过且没修改代码，可以跳过）。
2. 双击项目文件夹里的 `run_calculator.bat` 文件，或者在 VSCode 的终端里输入 `./simplecalculator.exe` 运行。

学习建议

1. **理解基本概念**：
   - 先理解 C 语言的基本语法和 GTK 的基本概念

- 熟悉结构体、函数、数组等基本知识点
2. **分析代码结构**：
   - 从 `main` 函数开始，了解程序的执行流程
   - 理解各个函数的功能和调用关系
3. **动手实践**：
   - 修改代码，添加新功能，如小数点、正负号等
   - 尝试添加更多的数学函数，如平方根、三角函数等
4. **查阅文档**：
   - 查阅 GTK 官方文档，了解更多 GTK 控件和功能
   - 学习 C 语言的高级特性，如内存管理、指针等
5. **代码风格**：
   - 注意代码的缩进和命名规范
   - 添加适当的注释，提高代码可读性
   - 学习如何编写模块化、可维护的代码

分阶段学习任务（供参考）

将学习过程细化为 5 个可落地的阶段，循序渐进地掌握 C 语言和 GTK 编程：

阶段 1：跑通项目（基础入门）

目标：配置环境，编译并运行程序，理解每个文件的作用。

具体任务：

1. 按照上述“GTK3 环境搭建”步骤完成环境配置。
2. 双击 `run_calculator.bat` 运行程序，测试四则运算功能。
3. 对照“项目结构”部分，理解每个文件的作用。

阶段 2：小改界面（熟悉 GTK 控件属性）（本任务可选）

目标：修改窗口标题、按钮颜色、显示字体等，熟悉 GTK 控件属性设置。

具体任务：

1. **修改窗口标题**：

在 `main` 函数中，找到 `gtk_window_set_title` ，将标题改为“我的计算器”：

```Plain Text
gtk_window_set_title(GTK_WINDOW(window), "我的计算器");
```

2. **修改显示字体**：

使用 `gtk_widget_override_font` 设置显示标签的字体（如微软雅黑、16 号）：

```Plain Text
// 在创建state.display后添加
PangoFontDescription *font_desc = pango_font_description_from_string("Microsoft YaHei 16");
gtk_widget_override_font(state.display, font_desc);
pango_font_description_free(font_desc); // 释放字体描述
```

3. **修改按钮颜色**：

使用 CSS 样式表设置按钮背景色（如数字按钮为浅灰，运算符按钮为蓝色）：

```Plain Text
// 在main函数开头（gtk_init之后）添加CSS加载代码
GtkCssProvider *css_provider = gtk_css_provider_new();
gtk_css_provider_load_from_data(css_provider,
    "button { background-color: #e0e0e0; }"  // 数字按钮浅灰
    "button.operator { background-color: #2196F3; color: white; }", // 运算符按钮蓝底白字
    -1, NULL);
gtk_style_context_add_provider_for_screen(gdk_screen_get_default(),
    GTK_STYLE_PROVIDER(css_provider), GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);

// 在创建运算符按钮时，添加CSS类
GtkWidget *add_button = create_button("+", G_CALLBACK(on_operator_clicked), &state);
gtk_style_context_add_class(gtk_widget_get_style_context(add_button), "operator");
// 对sub_button、mul_button、div_button做同样处理
```

4. 重新编译并运行，查看界面变化。

阶段 3：添加功能（拓展核心逻辑）（本任务代码请自行完善）

目标：添加小数点、正负号、退格键功能，完善计算器实用性。

具体任务：

1. **添加小数点功能**：
   - 在 `main` 函数中创建小数点按钮，连接回调函数 `on_decimal_clicked`。
   - 实现 `on_decimal_clicked` 函数，避免重复输入小数点：

Plain Text

```
void on_decimal_clicked(GtkButton *button, gpointer data) {
    (void)button;
    CalculatorState *state = (CalculatorState *)data;
    if (state->new_number) {
        strcpy(state->current_number, "0.");
        state->new_number = 0;
    } else if (strchr(state->current_number, '.') == NULL) {
        // 若当前数字无小数点，则追加
        strcat(state->current_number, ".");
    }
    update_display(state);
}
```

   - 将小数点按钮添加到网格（如第 4 行第 2 列）。
2. **添加正负号功能**：
   - 创建“+/-”按钮，连接回调函数 `on_sign_clicked`。
   - 实现 `on_sign_clicked` 函数，切换数字正负：

Plain Text

```
void on_sign_clicked(GtkButton *button, gpointer data) {
    (void)button;
    CalculatorState *state = (CalculatorState *)data;
    if (strlen(state->current_number) > 0) {
        double current = atof(state->current_number);
        current = -current;
        snprintf(state->current_number, sizeof(state->current_number), "%.8g", current);
        update_display(state);
    }
}
```

3. **添加退格键功能：**
   - 创建“←”按钮，连接回调函数 `on_backspace_clicked`。
   - 实现 `on_backspace_clicked` 函数，删除最后一个字符：

Plain Text

```
void on_backspace_clicked(GtkButton *button, gpointer data) {
    (void)button;
    CalculatorState *state = (CalculatorState *)data;
    int len = strlen(state->current_number);
    if (len > 0) {
        state->current_number[len - 1] = '\0';
        if (len == 1) {
            // 若删除后为空，显示0
            strcpy(state->current_number, "0");
            state->new_number = 1;
        }
        update_display(state);
    }
}
```

4. 重新编译并测试新功能。

阶段 4：提升健壮性（修复潜在问题）（本任务代码请自行完善）

目标：修复除零错误、输入长度限制、连续运算符问题，提高代码稳定性。

具体任务：

1. **修复除零错误：**

在 `on_operator_clicked` 和 `on_equals_clicked` 的除法分支中添加除数为 0 的检查：

Plain Text

```
case '/':
    if (current == 0) {
        strcpy(state->current_number, "Error");
        update_display(state);
        state->operation = '\0';
        state->new_number = 1;
        return;
    }
    state->result /= current;
    break;
```

2. **限制输入长度：**

在 `on_number_clicked` 中追加数字前检查长度：

Plain Text

```
if (state->new_number) {
    strcpy(state->current_number, text);
    state->new_number = 0;
} else {
    if (strlen(state->current_number) < 20) { // 限制最多20位
        strcat(state->current_number, text);
    }
}
```

3. **修复连续运算符问题：**

在 `on_operator_clicked` 中，若已开始输入新数字（`new_number=1`），仅更新运算符，不重复计算：

```Plain Text
if (strlen(state->current_number) > 0) {
    if (!state->new_number) { // 仅当未开始新数字时计算
        double current = atof(state->current_number);
        // ... 原有switch计算逻辑 ...
    }
    state->operation = text[0];
    state->new_number = 1;
    // ... 原有显示逻辑 ...
}
```

4. 测试边界情况（如输入超长数字、连续点击运算符、除以 0）。

阶段 5：进阶拓展（深入学习）（本任务代码请自行完善）

目标：实现四则运算优先级、添加数学函数、保存历史记录，提升项目复杂度。

具体任务：

1. **实现四则运算优先级**：
   - 不再实时计算，而是将表达式（如“1+2*3”）存储为字符串。
   - 使用“栈”数据结构解析表达式，先算乘除后算加减（可参考“逆波兰表达式”算法）。
2. **添加数学函数**：
   - 创建“√”（平方根）、“x^2”（平方）、“%”（取模）按钮。
   - 实现相应回调函数，如平方根：

Plain Text

```
void on_sqrt_clicked(GtkButton *button, gpointer data) {
    (void)button;
    CalculatorState *state = (CalculatorState *)data;
    if (strlen(state->current_number) > 0) {
        double current = atof(state->current_number);
        if (current >= 0) {
            current = sqrt(current);
            snprintf(state->current_number, sizeof(state->current_number), "%.8g", current);
        } else {
            strcpy(state->current_number, "Error");
        }
        update_display(state);
        state->new_number = 1;
    }
}
```

3. **保存历史记录**：
   - 添加“历史”按钮，点击后弹出新窗口显示计算历史。
   - 使用文件操作（`fopen`、`fprintf`、`fscanf`）将历史记录保存到`history.txt`。
4. 重新编译并测试进阶功能，尝试优化代码结构。